

一、名词解释

I/O 硬件：指的是输入/输出设备，如键盘、鼠标、显示器、磁盘等，这些硬件用于与计算机进行数据传输。

I/O 软件：是一组程序和工具，用于管理和控制 I/O 硬件的操作，包括设备驱动程序、中断处理程序、缓冲管理以及相关的系统调用等。

系统调用处理程序：是操作系统内核中的一部分，负责处理来自用户空间的系统调用请求，实现用户程序与内核之间的交互。

系统调用服务例程：是系统调用处理程序调用的内核函数，用于实现具体的系统调用功能。

系统级 I/O 函数：是操作系统提供的用于执行 I/O 操作的 API（应用程序接口），如 C 语言中的 stdio 函数库。

虚拟文件系统（VFS）：是一种用于网络环境的分布式文件系统接口，它允许不同的文件系统与操作系统进行交互，而无需考虑具体的文件系统实现细节。

流缓冲区：用于提高 I/O 操作的效率和性能，通常用于在数据读取或写入时进行暂存，以减少系统调用的次数和内存访问的延迟。

索引节点：通常用于文件系统中，包含文件的元数据（如文件大小、权限、创建时间等），是文件系统用来定位和访问文件的关键数据结构。

系统打开文件表：是操作系统内核维护的一个数据结构，用于记录当前打开的文件及其相关信息，如文件描述符、文件状态等。

磁盘高速缓存：是操作系统用于提高磁盘 I/O 性能的一种技术，通过缓存最近访问过的磁盘数据块，减少对磁盘的物理访问次数。

程序直接控制 I/O：指用户程序直接通过特定的 I/O 指令或系统调用进行 I/O 操作，而不依赖于操作系统提供的 I/O 软件。

就绪状态：在多任务操作系统中，通常用于描述一个进程或线程已经准备好执行，等待被调度器分配到 CPU 上运行的状态。

多重中断：指的是在一个中断处理程序执行过程中，发生了另一个中断，需要暂时保存当前中断的上下文并处理新中断的情况。

中断嵌套：与多重中断类似，但更强调中断之间的层次关系，即一个中断可以在另一个中断的处理过程中被触发和处理。

设备控制器：是硬件设备与计算机之间的接口电路，负责控制和管理硬件设备的操作，如磁盘控制器、显卡控制器等。

I/O 端口：是 CPU 与外部设备通信的接口，用于发送和接收数据以及控制信号。

状态端口：是 I/O 端口的一种，用于读取或设置硬件设备的状态信息。

I/O 地址空间：是计算机内存中用于映射 I/O 端口和设备控制器的地址范围，使得 CPU 可以通过内存访问的方式与 I/O 设备进行通信。

存储器映射 I/O：是一种 I/O 技术，通过将 I/O 端口或设备控制器映射到内存地址空间，使得 CPU 可以像访问内存一样访问这些 I/O 设备。

I/O 指令：是 CPU 执行的指令，用于启动 I/O 操作，如读取或写入数据。

中断响应优先级：是操作系统用来决定哪个中断应该首先被响应的优先级规则。

中断处理优先级：是操作系统用来决定哪个中断处理程序应该首先被执行的优先级规则。

用户空间 I/O 软件：用户空间 I/O 软件是运行在操作系统用户空间中的程序，用于处理输入/输出 (I/O) 操作。这些软件通常通过系统调用来与内核空间 I/O 软件进行交互，从而实现了对硬件设备的访问和控制。

内核空间 I/O 软件：内核空间 I/O 软件是操作系统内核中负责处理 I/O 操作的部分。它管理硬件设备的驱动程序，处理中断，以及提供系统调用接口供用户空间程序使用。

设备驱动程序：设备驱动程序是一种软件，用于控制计算机上的硬件设备（如打印机、键盘、鼠标、音频/视频设备等），以便它们能够与计算机系统协调工作。设备驱动程序通常由设备制造商提供，并与操作系统紧密集成，以便系统可以识别和使用设备。

文件描述符：文件描述符是由无符号整数表示的句柄，进程使用它来标识打开的文件。文件描述符与包括相关信息（如文件的打开模式、文件的位置类型、文件的初始类型等）的文件对象相关联。

目录文件：目录文件是文件系统中的特殊文件，用于存储多个文件目录的信息。它包含了多个文件或目录的元数据，使得文件系统能够方便地管理和查找这些文件或目录。

高速缓存 RAM：高速缓存 RAM (Random Access Memory) 是一种用于暂时存储数据和程序的内存设备。它具有快速的读写能力，并通常用于存储 CPU 经常访问的数据，以提高数据访问速度。

中断控制 I/O：中断控制 I/O 是一种 I/O 控制方式，其中当外部设备准备好进行数据传输时，它会向 CPU 发送一个中断信号。CPU 在接收到中断信号后，会暂停当前的任务，转而执行中断处理程序来处理 I/O 操作。

DMA 方式：DMA (Direct Memory Access) 方式是一种允许某些硬件子系统（如磁盘控制器）直接访问主存，而无需通过 CPU 进行数据传输的技术。它提高了数据传输的效率，减少了 CPU 的介入。

控制端口：控制端口是计算机系统中用于发送和接收控制信号的端口。它通常用于与硬件设备通信，以控制设备的操作和行为。

独立编址方式：独立编址方式是指 I/O 端口地址和内存地址相互独立，使用不同的地址空间进行编址。这种编址方式使得 I/O 端口和内存可以互不干扰地进行访问。

中断服务程序：中断服务程序是操作系统内核中用于处理特定中断事件的程序。当中断发生时，CPU 会跳转到相应的中断服务程序执行，以处理中断事件。

文件元数据：文件元数据是关于文件自身的信息，如文件的创建时间、修改时间、大小、权限等。这些信息通常存储在文件系统的元数据结构中，用于管理和访问文件。

目录项：目录项是文件系统中用于记录文件或目录信息的条目。它包含了文件或目录的名称、属性以及其他相关元数据，使得用户可以方便地查找和访问这些文件或目录。

I/O 控制方式：I/O 控制方式是指计算机系统中用于管理 I/O 操作的方法和技术。常见的 I/O 控制方式包括程序直接控制、中断驱动、DMA 等。

中断屏蔽字：中断屏蔽字是一种用于控制中断响应的寄存器或变量。通过设置中断屏蔽字，可以屏蔽或允许特定的中断请求，以实现中断的优先级管理。

DMA 控制器：DMA 控制器是一种硬件设备，用于实现 DMA 方式的数据传输。它负责在内存和设备之间直接传输数据，而无需 CPU 的干预。

数据端口：数据端口是计算机系统中用于传输数据的端口。它通常用于与硬件设备通信，以发送和接收数据。

统一编址方式：统一编址方式是指将 I/O 端口地址和内存地址统一进行编址，使用相同的地址空间。这种编址方式简化了 I/O 操作的访问方式，但也可能导致内存空间的浪费。

可编程中断控制器：可编程中断控制器是一种用于管理中断请求的硬件设备。它允许操作系统对中断进行优先级排序、屏蔽和响应控制，以实现中断的高效处理。

中断请求寄存器：中断请求寄存器是用于记录当前待处理的中断请求的寄存器。当中断发生时，相应的中断请求会被记录到中断请求寄存器中，以供 CPU 或中断控制器处理。

二、简答题

(1) I/O 子系统的层次结构是怎样的？

解析：I/O 子系统的层次结构通常从上至下依次为：用户层 I/O 软件、设备独立性软件、设备驱动软件、中断处理软件和硬件（包括机械部件和电子部件）。这种层次结构的设计使得 I/O 操作更加模块化和抽象化，方便了软件的开发和维护。

(2) 系统调用封装函数对应的机器级代码结构是怎样的？

解析：系统调用封装函数对应的机器级代码结构通常包括若干条传送指令后跟一条陷阱指令。传送指令用于传递系统调用的参数，而陷阱指令（如 `int $0x80`）则用于使 CPU 陷入内核进行处理。这种结构是系统调用在机器层面的实现方式。

(3) 为什么系统调用的开销很大？

解析：系统调用的开销很大，主要是因为系统调用涉及从用户态切换到内核态的过程，这需要时间进行上下文切换。此外，在内核态执行时，CPU 工作在管理员模式，这涉及到特殊的堆栈和内存环境，需要在系统调用开始时建立并在结束后恢复。这种运行环境的切换以及内核态下对资源的访问控制都增加了系统调用的开销。

(4) C 标准 I/O 库函数是在用户态执行还是在内核态执行？

解析：C 标准 I/O 库函数（如 `printf` 和 `scanf`）是在用户态执行的。这些函数提供了用户程序与 I/O 设备之间的接口，但实际的 I/O 操作（如读写数据）是由内核在内核态下完成的。用户态的程序通过系统调用请求内核进行这些操作。

(5) 与 I/O 操作相关的系统调用封装函数是在用户态执行还是内核态执行？

解析：与 I/O 操作相关的系统调用封装函数是在内核态执行的。当用户程序需要执行 I/O 操作时，它会通过系统调用接口请求内核执行相应的 I/O 操作。内核会切换到内核态，执行相应的系统调用封装函数，完成 I/O 操作后再返回到用户态。

(6) 什么是程序直接控制 I/O 方式？说明其工作原理。

解析：程序直接控制 I/O 方式是指 CPU 直接通过 I/O 指令对 I/O 设备进行操作的方式。在这种方式下，CPU 会向 I/O 设备发送控制命令和数据，然后等待设备完成操作并返回状态信息。这种方式的缺点是 CPU 需要频繁地检查设备的状态，降低了 CPU 的效率。

(7)为什么在保护现场和恢复现场的过程中，CPU 必须关中断？

解析：在保护现场和恢复现场的过程中，CPU 必须关中断，这是因为中断会打断当前的执行流程，如果允许中断发生，可能会干扰正在处理中的操作系统的信息，导致恢复现场时状态不一致，甚至发生错误。关中断可以确保在恢复现场之前，不会有其他中断打断当前的处理过程。

(8)什么是中断控制 I/O 方式?说明其工作原理。

解析：中断控制 I/O 方式是一种通过中断机制进行 I/O 操作的方式。当 I/O 设备准备好进行数据传输时，它会向 CPU 发送一个中断信号。CPU 在接收到中断信号后，会暂停当前的任务，转而执行中断处理程序来处理 I/O 操作。这种方式可以实现 CPU 与 I/O 设备之间的并行操作，提高了系统的效率。

(9)DMA 方式能够提高成批数据交换效率的主要原因何在？

解析：DMA 方式能够提高成批数据交换效率的主要原因在于它允许数据直接在内存和设备之间进行传输，而无需 CPU 的介入。这种方式减少了 CPU 的开销，使得 CPU 可以专注于执行其他任务。同时，DMA 方式还可以实现高速的数据传输，进一步提高了数据交换的效率。

(10)DMA 控制器在什么情况下发出中断请求信号？

解析：DMA 控制器在数据块传输完毕时发出中断请求信号。这是因为 DMA 控制器负责在内存和设备之间直接传输数据块，当一块数据传输完成后，它需要通知 CPU 以便 CPU 可以进行后续的处理或启动新的数据传输任务。通过发出中断请求信号，DMA 控制器可以确保 CPU 能够及时响应并处理数据传输的完成情况。

(11) I/O 端口的编址方式有哪两种?各有何特点？

解析：I/O 端口的编址方式主要有独立编址和统一编址两种。独立编址方式下，I/O 端口地址和内存地址相互独立，使用不同的地址空间进行编址。这种方式使得 I/O 端口和内存可以互不干扰地进行访问。统一编址方式下，I/O 端口地址和内存地址统一进行编址，使用相同的地址空间。这种方式简化了 I/O 操作的访问方式，但可能导致内存空间的浪费。

(12)为什么中断控制器把中断类型号放 I/O 总线的数据线上而不是放在地址线上？

解析：中断控制器把中断类型号放在 I/O 总线的数据线上而不是放在地址线上的原因主要是为了适应中断处理机制。中断类型号标识了发生中断的具体原因或来源，它需要通过 I/O 总线传递给 CPU。将数据线用于传输中断类型号可以确保数据在传输过程中的准确性和可靠性。而地址线主要用于在内存或 I/O 设备中定位特定的存储单元或端口，其传输的数据与中断处理不直接相关。因此，将中断类型号放在数据线上是更合理的选择。